

第二章 进程管理

by Jiacheng Xu

hint: 章节2.4, 写的比较模糊, 后续需要改进

- 第二章 进程管理
 - 2.1 进程的基本概念
 - 2.1.1 前趋图
 - (1) 定义
 - (2) 组成和描述方法
 - 2.1.2 进程的顺序执行
 - (1) 顺序执行的基本概念 (3种执行方式)
 - (2) 顺序执行的特征 (3种特征, 注意理解)
 - (3) 顺序执行存在的问题
 - 2.1.3 程序的并发执行
 - (1) 并发执行的基本概念
 - (2) 并发执行存在的问题 (描述不可再现性)
 - (3) 程序并发执行的条件 (理解, 可忽略)
 - (4) 并发执行的特征 (3个)
 - 2.1.4 进程的特征与状态
 - (1) 进程的定义
 - (2) 进程的特征
 - (3) 进程的三种基本状态
 - 2.1.5 进程控制块 (PCB)
 - (1) 相关概念
 - (2) 内容
 - (3) 组织形式
 - 2.2 进程控制 (理解过程即可)
 - 2.2.1 进程的创建create
 - (1) 原语
 - (2) 引起创建的事件
 - (3) 创建流程
 - (4) 进程创建树
 - 2.2.2 进程的终止/撤销 exit
 - (1) 原语
 - (2) 引起终止/撤销的事件

- (3) 终止/撤销流程
- 2.2.3 进程的阻塞 block
 - (1) 原语
 - (2) 引起阻塞的事件
 - (3) 阻塞原理
- 2.2.4 进程的唤醒 wakeup
 - (1) 原语
 - (2) 引起唤醒的事件
 - (3) 唤醒原理
- 2.2.5 进程的挂起suspend与激活activate
 - (1) 原语
 - (2) 引起挂起的事件
 - (3) 引起激活的事件
- 2.3 进程同步
 - 2.3.1 进程同步的基本概念
 - (1) 进程同步问题产生原因
 - (2) 临界资源
 - (3) 临界区
 - (4) 同步机制遵循的原则
 - 2.3.2 实现进程同步的早期方法（考的概率不大）
 - (1) 软件解法1：按需访问
 - (2) 软件解法2：轮询
 - (3) 软件解法3：访前先看
 - (4) 软件解法4：Peterson算法
 - (5) 硬件解法1：关中断
 - (6) 硬件解法2：Test-and-Set指令实现互斥
 - (7) 硬件解法3：Swap指令
 - (8) 特点总结
 - 2.3.3 信号量机制
 - (1) 整数型信号量
 - (2) 记录型信号量
 - 2.3.4 信号量的应用
 - (1) 信号量用于互斥
 - 1. 堆栈入栈问题
 - 2. 过独木桥问题
 - (2) 信号量用于同步（协作过程）
 - 1. 双进程打印

- 2. 三进程打印
- 2.4 经典进程同步问题
 - 2.4.1 生产者-消费者问题
 - (1) 问题描述
 - (2) 相关各类问题
 - 1. 单生产者单消费者、多缓冲
 - 2. 多生产者多消费者、多缓冲
 - 3. 多生产者多消费者、单缓冲
 - 4. 单生产者单消费者 单缓冲
 - 5. 允许生产者写时，消费者可读
 - 6. 缓冲池无限大(?)
 - 7. 调整生产者wait顺序(?)
 - 8. 调整signal顺序(?)
 - 9. 每个消费者消费每个消息1次？如何实现？
 - 2.4.2 读者-写者问题
 - (1) 问题描述
 - (2) 解答
 - 2.4.3 哲学家进餐问题
 - (1) 问题描述
 - (2) 解答
 - 2.4.5 信号量集
 - (1) AND型信号量集机制
 - (2) 一般信号量集机制
- 2.5 进程通信
 - 2.5.0 定义
 - 2.5.1 共享存储器系统
 - 2.5.2 消息通信系统
 - (0) 基本定义
 - (1) 直接通信方式
 - (2) 间接通信方式（使用信箱通信）
 - 1. 使用信箱进行通信的原理
 - 2. 信箱
 - 2.5.3 管道（Pipe）通信系统
- 2.6 线程-函数级并发执行
 - 2.6.1 线程（Thread）的基本概念
 - (1) 线程的引入
 - (2) 线程与进程的关系

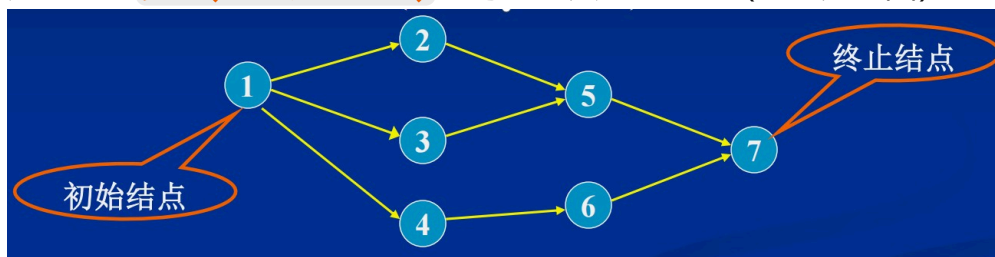
- (3) 线程的开销
- (4) 引入进程的好处
- 2.6.2 线程的实现方法
 - (1) 内核支持线程
 - (2) 用户级线程

2.1 进程的基本概念

2.1.1 前趋图

(1) 定义

用于描述 **实体(进程、程序段)** 之间执行次序的DAG (有向无环图)。



(2) 组成和描述方法

两个组成部分: **结点和有向边**

结点: 代表一条语句, 一段程序或一个进程

有向边: 两结点之间存在的 **先后次序关系**, 有向边集合

实例 (上图) 注意掌握描述方法

$P = \{P1, P2, P3, P4, P5, P6, P7\}$

有向边集合 = $\{ (P1, P2), (P1, P3), (P1, P4), (P2, P5), (P3, P5), (P4, P6), (P5, P7), (P6, P7) \}$

2.1.2 进程的顺序执行

(1) 顺序执行的基本概念 (3种执行方式)

三种执行方式, 主要区别是指令层级

1. **程序段**级顺序执行: 分三段程序段执行, 输入I, 计算C, 打印P
2. **程序**级顺序执行: 多个程序执行, 每个程序有各自的ICP
3. **语句**级顺序执行: 以语句为基本单位执行

(1) 顺序执行的基本概念

■程序段级顺序执行

□输入程序段: I

□计算程序段: C

□打印程序段: P



■程序级顺序执行



■语句级顺序执行

S1: $a := x + y$

S2: $b := a - 5$

S3: $c := b + 1$



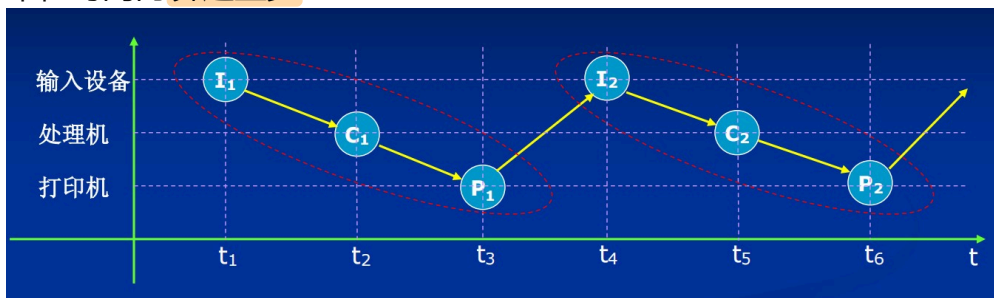
合肥工业大学操作系统课程组 2014

(2) 顺序执行的特征 (3种特征, 注意理解)

1. **顺序性**: 处理机严格按照程序规定的顺序执行程序
2. **封闭性**: 程序独占全机资源, 不受其他程序的影响
3. **可再现性**: 程序执行结果 = 初始条件 + 执行时的环境

(3) 顺序执行存在的问题

- 整体资源利用率低
- 单位时间内算题量少



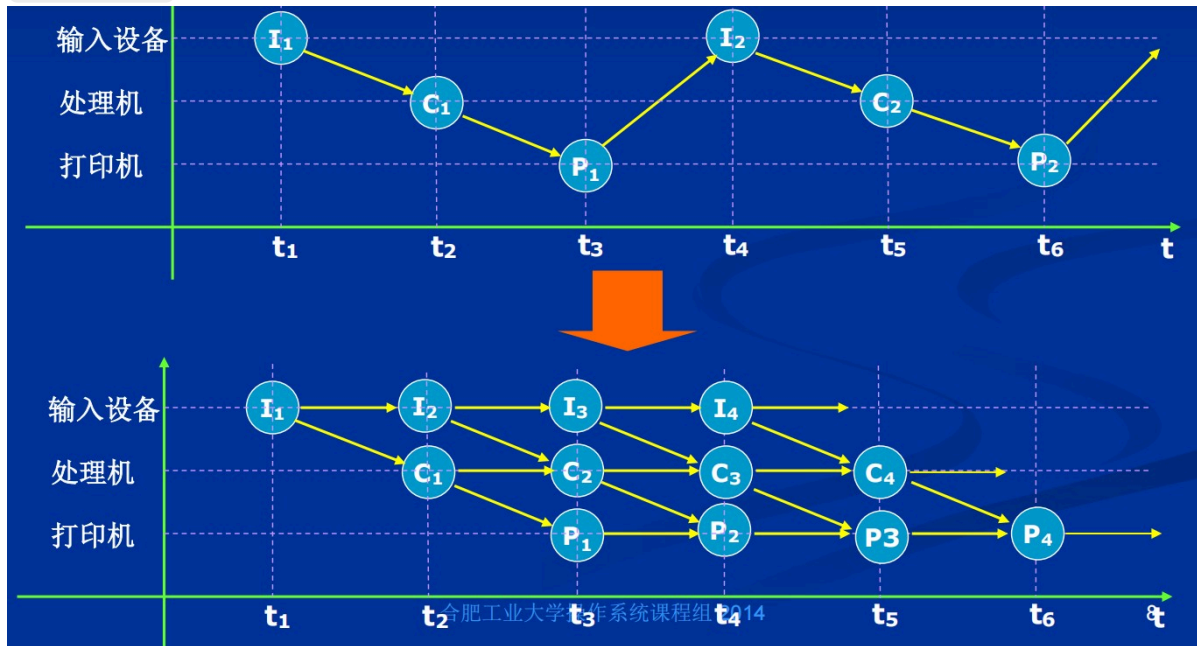
❗ 上图是如何反应问题的

三个设备依次执行, 无法实现并发
回忆CPU流水, 纯顺序执行导致效率低且单位时间完成的任务量少

2.1.3 程序的并发执行

(1) 并发执行的基本概念

使用的技术：多道程序设计技术（类似CPU多周期流水）



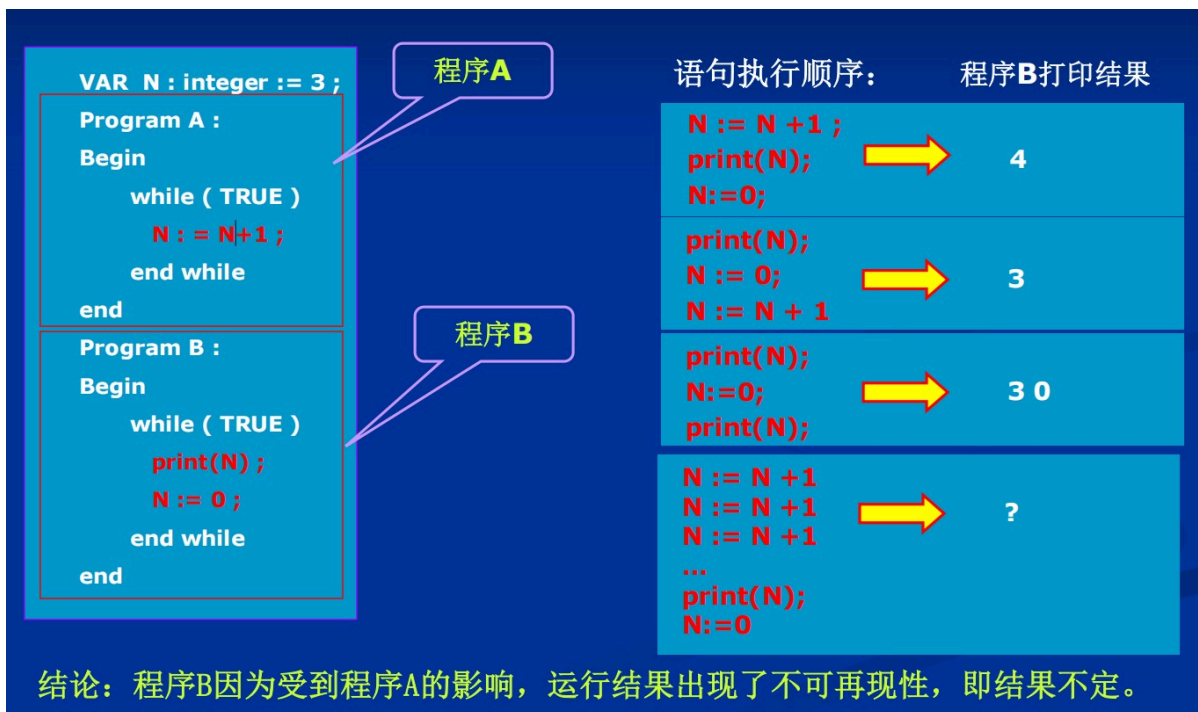
(2) 并发执行存在的问题（描述不可再现性）

不可再现性：程序执行结果受其他程序的影响，**结果不定**

解释：

- 图中的两个程序一起执行，**执行的顺序是不确定的**，输出的结果也可能不一样，无法预估
- 反观顺序执行，明确了两个程序的执行顺序，也就可以确定输出结果

直观理解：多周期流水CPU数据冲突！



(3) 程序并发执行的条件 (理解, 可忽略)

要求满足 **Bernstein条件**

✓ Bernstein条件

若两个程序p1和p2能满足下述条件,他们便能并发执行,且具有再现性:

$$(R(p1) \cap W(p2)) \cup ((R(p2) \cap W(p1)) \cup (W(p1) \cap W(p2))) = \emptyset$$

其中R集合为程序p需要 **读取的变量集合**, W集合为程序p需要 **写入的变量集合**

(4) 并发执行的特征 (3个)

1. **间断性**: 资源共享, 导致程序执行具有 "执行-暂停-执行" 的特点
2. **失去封闭性**: 程序不再独占全机资源, 运行受到其他程序的影响
3. **不可再现性**: 程序执行结果受其他程序的影响, 结果不定

2.1.4 进程的特征与状态

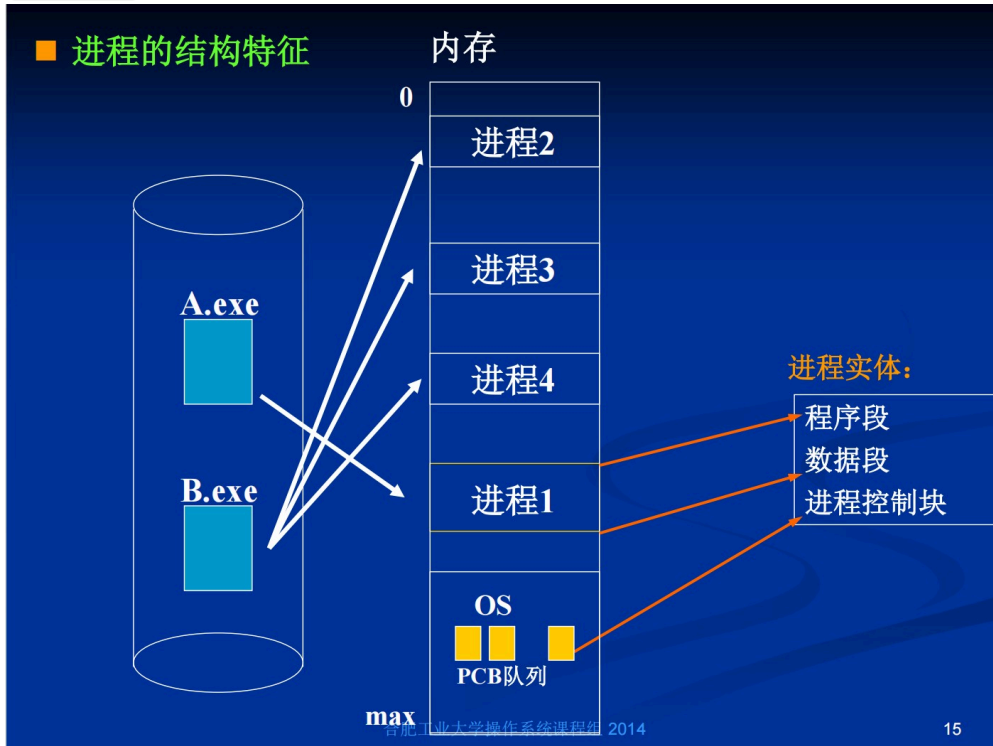
(1) 进程的定义

进程实体的 **运行** 过程, 是系统进行 **资源分配和调度的独立单位**

(2) 进程的特征

1. **动态性**: 是程序的一次 **执行过程**, 因而是动态的。
2. **并发性**: 引入进程就是为了和其他程序 **并发执行**, 提高资源利用率。

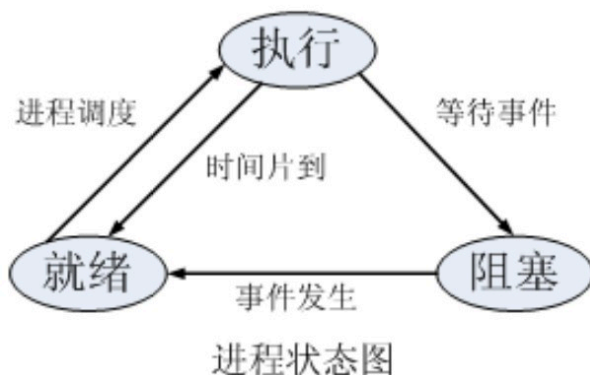
3. **独立性**：能**独立运行**的基本单位，也是**资源分配和调度的基本单位**。
4. **异步性**：进程以各自**独立的、不可预知的速度**向前推进。
5. **结构特征**：程序段、数据段、进程控制块（PCB）



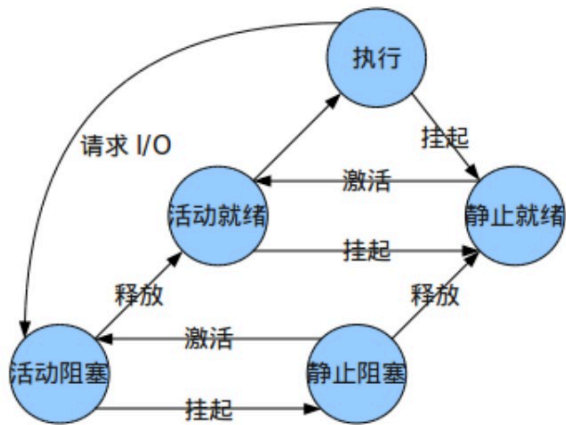
(3) 进程的三种基本状态

状态	含义
就绪状态	进程已经获得除处理机之外的所有资源，一旦获得处理机就可以立即执行
执行状态	一个进程已获得了必要的资源，正在处理机上运行
阻塞状态	正在执行的进程，由于发生某事件而暂停无法继续执行（如等待I/O、数据到达）

进程之间的转化见下图（一般考这个）



上图为三个基本状态之间的转化，下图加入了两个状态，并添置了挂起状态



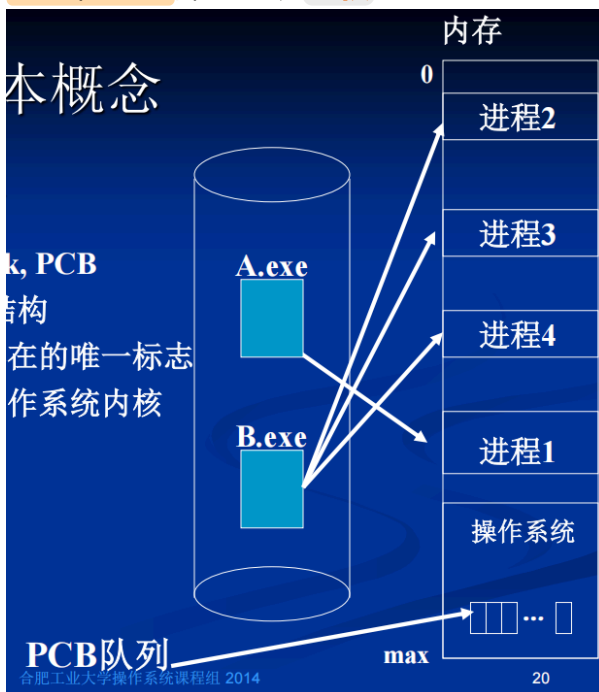
2.1.5 进程控制块 (PCB)

(1) 相关概念

1. 定义:

- Process Control Block, PCB
- OS 管理进程的数据结构
- 进程控制块是进程 **存在的唯一标志**

2. PCB位置: 操作系统 **内核**

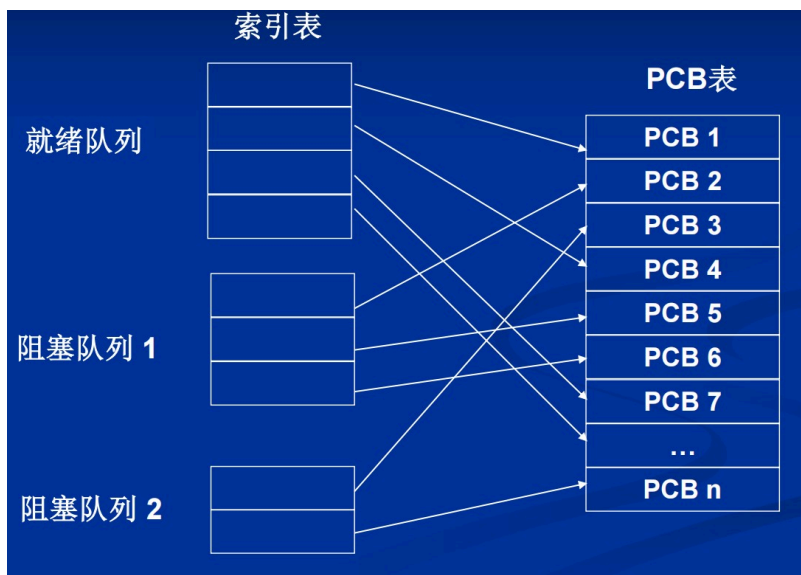
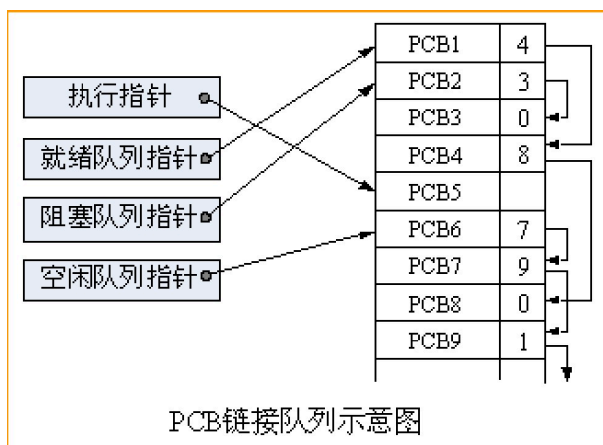


(2) 内容

- 进程标志符
- 处理机状态
- 进程调度信息
- 进程控制信息

(3) 组织形式

组织形式	方式
线性方式	直接将PCB线性相连
链接方式	使用多个指针，将状态相同的PCB链接成一个队列
索引方式	根据不同状态建立索引表，表中存放PCB地址



2.2 进程控制（理解过程即可）

2.2.1 进程的创建create

(1) 原语

create()

(2) 引起创建的事件

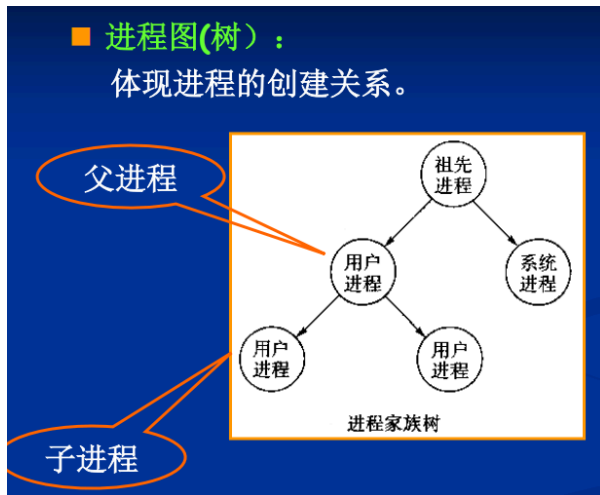
- 用户登录

- 作业调度
- 提供服务
- 应用请求

(3) 创建流程

1. 申请空白PCB，没有则无法建立
2. 有空白PCB，设置PCB的初始值
3. 设置进程状态为就绪，插入就绪队列

(4) 进程创建树



2.2.2 进程的终止/撤销 exit

(1) 原语

exit()

(2) 引起终止/撤销的事件

- 正常结束
- 异常结束
- 外界干预

(3) 终止/撤销流程

1. 检索需要终止的进程的PCB
2. 如果当前进程处于执行状态，**终止执行**；如果还有**子孙进程**，也需要终止
3. 终止进程所有资源 归还 **父进程or系统**
4. 终止进程or其PCB 移出当前所在**队列**

2.2.3 进程的阻塞 block

(1) 原语

block()

(2) 引起阻塞的事件

1. 请求系统服务
2. 请求操作
3. 等待数据、资源
4. 等待工作任务

(3) 阻塞原理

进程自己阻塞自己

2.2.4 进程的唤醒 wakeup

(1) 原语

wakeup()

(2) 引起唤醒的事件

1. 允许系统服务
2. 允许操作
3. 数据、资源到达
4. 工作任务到达

(3) 唤醒原理

由其他进程唤醒

2.2.5 进程的挂起suspend与激活activate

(1) 原语

挂起：suspend()

激活：activate()

(2) 引起挂起的事件

1. 调试系统
2. 暂停程序执行

(3) 引起激活的事件

1. 调试系统结束
2. 程序继续执行

2.3 进程同步

2.3.1 进程同步的基本概念

(1) 进程同步问题产生原因

- 多进程并发执行
- 进程共享系统资源，部分资源可共享，部分资源**仅可独占**（关键是独占）
- 因此产生各种**进程间相互制约**的关系
 1. **间接相互制约关系**：共享某种系统资源
 2. **直接相互制约关系**：进程间合作

(2) 临界资源

定义：**一次**仅能为**一个进程**所使用的资源（仅可独占）

举例：硬件（打印机）、软件（内存变量、指针、数组）

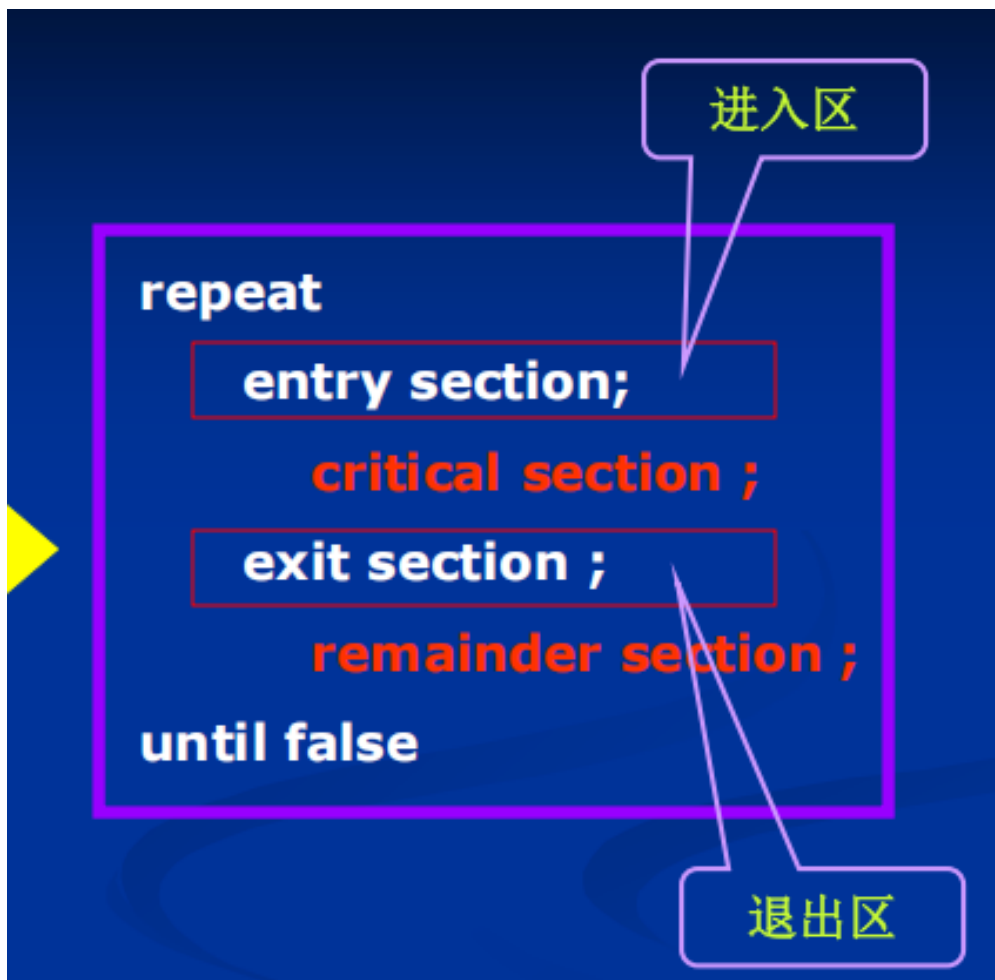
(3) 临界区

定义：进程中**访问临界资源**的代码段

实现临界资源互斥访问：临界区访问互斥

临界区访问控制模型

进入区→临界区→退出区



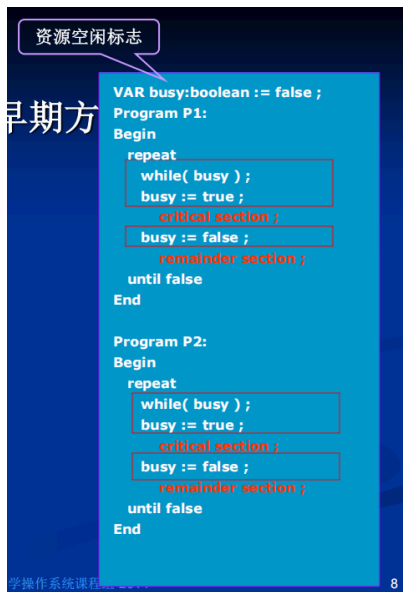
(4) 同步机制遵循的原则

1. **空闲让进**：当无进程处于临界区时，请求进入临界区的进程可立即进入
2. **忙则等待**：当已有进程进入临界区时，其他试图进入临界区进程须等待
3. **有限等待**：对要求访问临界资源进程，保证能在有限时间内进入临界区
4. **让权等待**：当进程不能进入临界区时，应释放处理机

2.3.2 实现进程同步的早期方法（考的的概率不大）

(1) 软件解法1：按需访问

方法：使用busy标志标记临界资源是否空闲

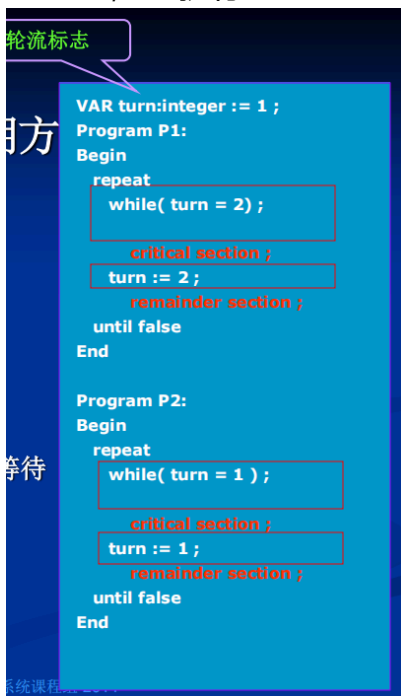


缺点： 违背忙则等待、有限等待、让权等待

(2) 软件解法2：轮询

方法： 使用turn定义进程执行顺序

- 如图，turn=1, P1执行；
- turn=2, P2执行



缺点： 严格限制资源访问顺序；违背有限等待和让权等待

(3) 软件解法3：访前先看

方法： 使用两个愿望标志pturn和qturn判断哪个进程需要访问

- pturn=true, 给P
- pturn=false, 给Q

```

VAR pturn,qturn:boolean := false,false ;
Program P:
Begin
  repeat
    pturn := true ;
    while( qturn ) ;
    critical section ;
    pturn := false ;
    remainder section ;
  until false
End

Program Q:
Begin
  repeat
    qturn := true ;
    while( pturn ) ;
    critical section ;
    qturn := false ;
    remainder section ;
  until false
End

```

缺点：违背空闲让进、有限等待、让权等待

(4) 软件解法4: Peterson算法

2.3 进程同步

2.3.2 实现进程同步的早期方法

(4) 软件解法4: Peterson算法,1981

```

#define FALSE 0
#define TRUE 1
#define N 2 // 进程的个数
int turn; // 轮到谁?
int interested[N]; // 兴趣数组, 初始值均为FALSE

void enter_region (int process) // process = 0 或 1
{
  int other; // 另外一个进程的进程号
  other = 1 - process;
  interested[process] = TRUE; // 表明本进程感兴趣
  turn = process; // 设置标志位
  while( turn == process && interested[other] == TRUE);
}

void leave_region (int process)
{
  interested[process] = FALSE; // 本进程已离开临界区
}

```

```

Program P1:
Begin
  repeat
    enter_region(0);
    critical section ;
    leave_region(0);
    remainder section ;
  until false
End

Program P2:
Begin
  repeat
    enter_region(1);
    critical section ;
    leave_region(1);
    remainder section ;
  until false
End

```

(5) 硬件解法1: 关中断

方法：进入锁之前关闭中断，完成锁测试并上锁后打开中断

(6) 硬件解法2: Test-and-Set指令实现互斥

C

```
boolean TS(boolean*lock){
    Boolean old;
    old = *lock;
    *lock = TRUE;
    return old;
}

do{
    ...
    while TS(&lock); // lock用于测试是否可以进入临界区
    critical section; // 临界区
    lock := FALSE; // 使用完成后释放lock
    remainder section;
}
```

方法：使用lock测试控制进入临界区

(7) 硬件解法3: Swap指令

2.3 进程同步

2.3.2 实现进程同步的早期方法

(6) 硬件解法2: SWAP指令

```
// 交换锁lock和key的值
function SWAP(lock, key ) {
    var tmp : boolean := lock ;
    lock := key ;
    key := tmp ;
}

function enter_region( var lock : boolean )
Var key:boolean ; //局部变量
Begin
    key := true;
    while(key)
        SWAP(lock,key);
end

function leave_region( var lock : boolean )
Begin
    lock := false;
end
```

```
VAR lock :boolean := false ;
Program P1:
Begin
    repeat
        enter_region(lock );
        critical section ;
        leave_region(lock);
        remainder section ;
    until false
End

Program P2:
Begin
    repeat
        enter_region(lock);
        critical section ;
        leave_region(lock);
        remainder section ;
    until false
End
```

方法：交换key和lock的值

(8) 特点总结

- **软件解法**：
 - 忙等待
 - 实现复杂
- **硬件解法**：
 - 简单有效、适用于多处理机
 - 忙等待

2.3.3 信号量机制

(1) 整数型信号量

定义：

1. 整数信号量S
2. 两个原子操作wait/signal(P/V)

原则：

- 除初始化外，S只能由wait/signal访问

方法：

- **wait()**
 - $S \leq 0$ 时不执行操作
 - 其他情况下， $S = S - 1$
- **signal()**
 - $S = S + 1$

2.3.3 信号量机制

(1) 整数型信号量，Dijkstra,1965

■ 定义

- 整数信号量S;
- 两个原子操作: wait/signal(P/V)

■ 原则:

除初始化外，S只能由wait/signal访问

```
VAR S:integer := 1 ;
function Wait ( var S: integer )
Begin
    while ( S <= 0 ) do no_op();
    S := S-1;
end

function Signal( var S : integer )
Begin
    S := S+1;
end
```

```
VAR S:integer := 1 ;
Program P1:
Begin
    repeat
        wait(S);
        critical section ;
        signal(S);
        remainder section ;
    until false
End
```

```
Program P2:
Begin
    repeat
        wait(S);
        critical section ;
        signal(S);
        remainder section ;
    until false
End
```

(2) 记录型信号量

定义：

- 记录型信号量S
- 两个原子操作wait/signal(P/V)
- 定义数据结构semaphore，存放资源数量value和阻塞进程队列L

```
type semaphore = record
    value: integer ;           // 资源数量
    L: list of process ;      // 阻塞进程队列
end
```

原则：

- 除初始化外，S只能由wait/signal访问

方法：

- wait()
 - 含义：申请一个单位资源
 - 没得到资源前，会一直阻塞，直到可以申请资源
 - $S.value = S.value - 1$
 - $S.value < 0$ 时，阻塞进程，放弃处理机，将当前进程插入S.L中
- signal()
 - 含义：释放一个单位资源
 - $S.value = S.value + 1$
 - $S.value \leq 0$ 时，唤醒S.L链表中第一个等待的进程（一定注意是小于等于 $\leq$ ）

(2) 记录型信号量

■ 定义

- 记录型信号量S；
- 两个原子操作：wait/signal(P/V)

■ 原则：

除初始化外，S只能由wait/signal访问

```
type semaphore = record
    value: integer ;           // 资源数量
    L: list of process ;      // 阻塞进程队列
end
```

```
function wait ( var S : Semaphore )
Begin
    S.value := S.value-1;
    if ( S.value < 0 ) then block (S.L);
end
function signal(var S : Semaphore )
Begin
    S.value := S.value+1;
    if ( S.value <= 0 ) then wakeup (S.L);
end
```

```
VAR S:semaphore:= 1 ;
Program P1:
Begin
    repeat
        wait(S);
        critical section ;
        signal(S);
        remainder section ;
    until false
End

Program P2:
Begin
    repeat
        wait(S);
        critical section ;
        signal(S);
        remainder section ;
    until false
End
```

2.3.4 信号量的应用

(1) 信号量用于互斥

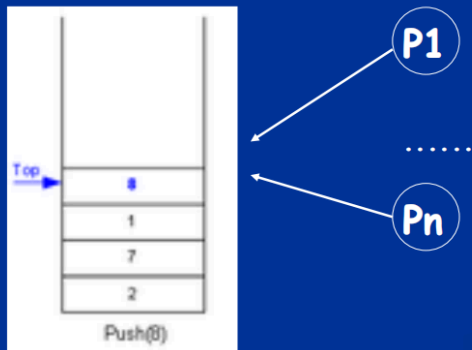
1. 堆栈入栈问题

2.3 进程同步

2.3.4 信号量的应用

(1) 信号量用于互斥

例1-扩展: n 个进程进行堆栈的入栈



合肥工业大学操作系统课程组 2

```
Parbegin
  VAR top : integer = -1 ;
      stack: array[0..n-1] of item;
      mutex: semaphore :=1 ;

  Process P1 :
  Begin
    while ( TRUE )
      wait( mutex ) ;
      top := (top+1) mod n;
      stack[top] := x ;
      signal(mutex) ;
    end while
  end

  .....

  Process Pn :
  Begin
    while ( TRUE )
      wait( mutex ) ;
      top := (top+1) mod n;
      stack[top] := y ;
      signal(mutex) ;
    end while
  End
Parend
```

核心代码：

- 提示: 其实wait和signal之间的操作不是很重要, 可以忽略

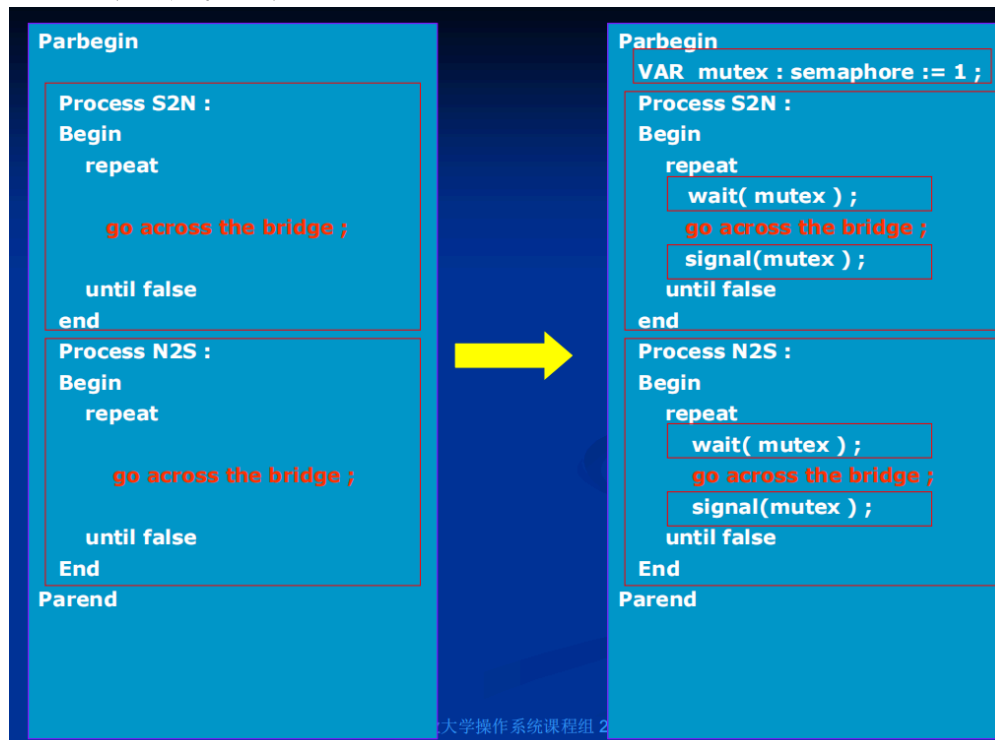
```

Process P1:
Begin
    while(TRUE)
        wait(mutex); // 关键
        top = (top+1) mod n;
        stack[top] := x;
        signal(mutex); // 关键
    End
...
Process Pn:
Begin
    while(TRUE)
        wait(mutex); // 关键
        top = (top+1) mod n;
        stack[top] := y;
        signal(mutex); // 关键
    End

```

2. 过独木桥问题

过独木桥问题，一次只能过一人



核心代码：

- 提示：其实wait和signal之间的操作不是很重要，可以忽略

C

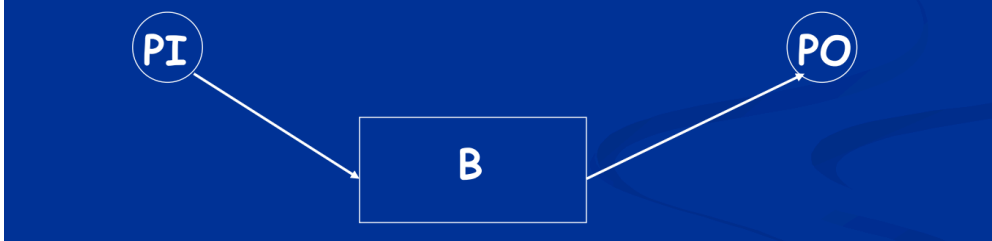
```
Process S2N:
Begin
    while(TRUE)
        wait(mutex); // 关键
        go across the bridge;
        signal(mutex); // 关键
    End

Process N2S:
Begin
    while(TRUE)
        wait(mutex); // 关键
        go across the bridge;
        signal(mutex); // 关键
    End
```

(2) 信号量用于同步（协作过程）

1. 双进程打印

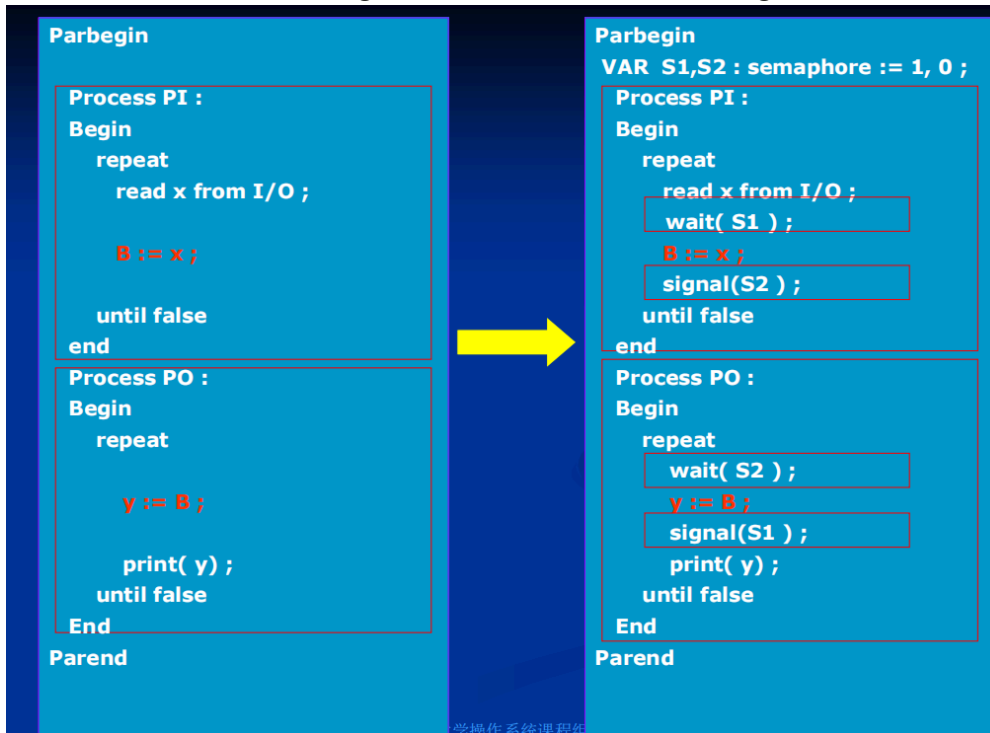
例3：PI,PO两进程共享缓冲区B,PI负责读入数据并保存在B中，PO负责打印B中数据，要求每个数据必须且只打印一次，不许漏打或重复打印。



解题思路：

- PI在访问B前读取
- PO在访问B后打印数据
- B使用信号量互斥访问
- 多信号量传递控制（不要写错了）：**
 - 设置两个信号量S1、S2，S1初值为1，S2初值为0

- PI读取数据在B后，需要signal(S2)激活PO读取（不能signal(S1)）
- PO打印完成后，需要signal(S1)激活PI继续（不能signal(S2)）



核心代码：

```

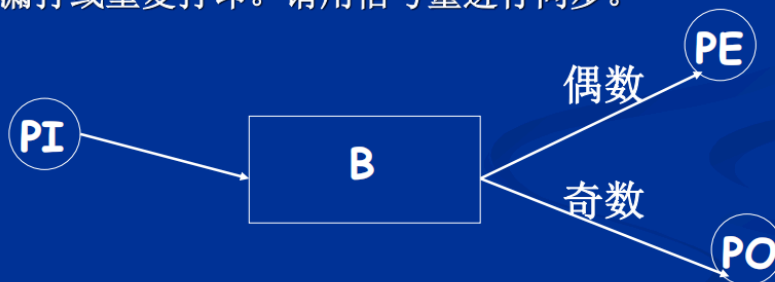
Process PI:
Begin
  repeat
    read x from I/O;
    wait(S1); // 关键
    B := x;
    signal(S2); // 关键
  until false
End

Process PO:
Begin
  repeat
    wait(S2); // 关键
    y := B;
    signal(S1); // 关键
    print(y);
  until false
End

```

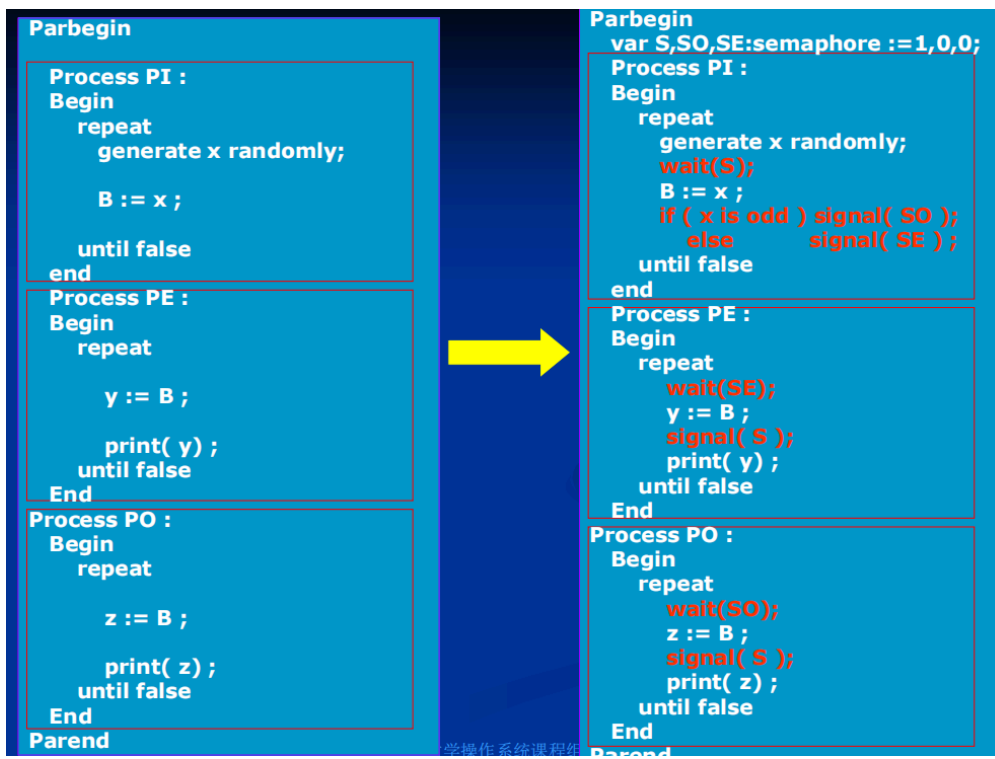
2. 三进程打印

例4: PI,PE,PO三进程通过共享缓冲区B协作,PI负责产生随机数并保存在B中,当B中数据为奇数时由PO负责打印,当B中数据为偶数时由PE负责打印,要求每个数据必须且只打印一次,不许漏打或重复打印。请用信号量进行同步。



解题思路：

- 根据读取的x控制信号量S即可，和1基本类似



核心代码：

```
Process PI:
Begin
    repeat
        generate x randomly
        wait(S);
        B := x;
        if(x is odd) signal(S0);
            else signal(SE);
    until false
End
```

```
Process PE:
Begin
    repeat
        wait(SE);
        y := B;
        signal(S);
        print(y);
    until false
End
```

```
Process P0:
Begin
    repeat
        wait(S0);
        z := B;
        signal(S);
        print(z);
    until false
End
```

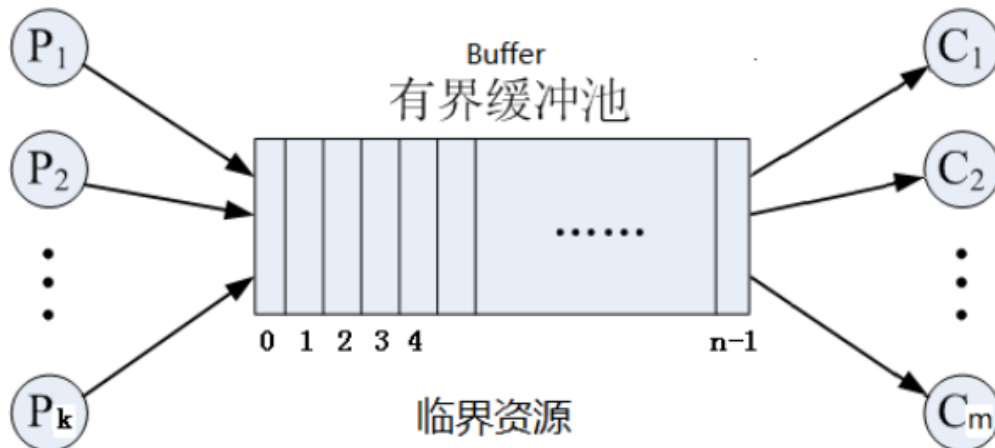
2.4 经典进程同步问题

2.4.1 生产者-消费者问题

(1) 问题描述

- 多个生产者、多个消费者共享n个缓冲区的缓冲池Buffer

- 每次生产/消费一个数据。
- 使用记录型信号量进行同步



(2) 相关各类问题

1. 单生产者单消费者、多缓冲

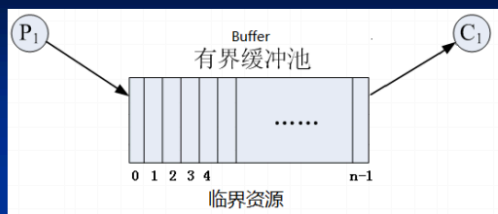
正常使用empty mutex full三个信号量
生产者

- wait(empty)保证不空
- wait(mutex)保证访问互斥
- signal(mutex)保证访问互斥
- signal(full)full信号+1

消费者反之

(2) 问题思考

■ 单生产者单消费者、多缓冲



■ 生产者-消费者标准程序

```

VAR mutex, empty, full: semaphore := 1, n, 0 ;
in,out: integer := 0, 0 ;
Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;
      wait( mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( mutex ) ;
      signal( full ) ;
    until false
  end
  Consumer:
  begin
    repeat
      wait( full ) ;
      wait( mutex ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( mutex ) ;
      signal( empty ) ;
      consume the item nextc ;
    until false
  end
Parend

```

2. 多生产者多消费者、多缓冲

正常使用empty mutex full三个信号量

(2) 问题思考

■ 多生产者多消费者、单缓冲



合肥工业大学操作

■ 生产者-消费者标准程序

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
    in,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;
      wait( mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( mutex ) ;
      signal( full ) ;
    until false
  end
  Consumer:
  begin
    repeat
      wait( full ) ;
      wait( mutex ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( mutex ) ;
      signal( empty ) ;
      consume the item nextc ;
    until false
  end
end
Parend
```

3. 多生产者多消费者、单缓冲

可以不使用mutex，因为只有1个缓冲池

(2) 问题思考

■ 多生产者多消费者、单缓冲



合肥工业大学操作系统

```
VAR empty, full: semaphore := 1, 0 ;
    in,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;

      Buffer(in) := nextp ;
      in := (in + 1) mod n ;

      signal( full ) ;
    until false
  end

  Consumer:
  begin
    repeat
      wait( full ) ;

      nextc = Buffer(out);
      out := (out + 1) mod n ;

      signal( empty ) ;
      consume the item nextc ;
    until false
  end
end
Parend
```

4. 单生产者单消费者 单缓冲

(2) 问题思考

■ 单生产者单消费者、单缓冲



■ 同例3

```
VAR empty, full: semaphore := 1, 0 ;
in, out: integer := 0, 0 ;
Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( full ) ;
    until false
  end

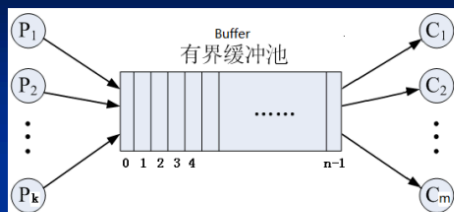
  Consumer:
  begin
    repeat
      wait( full ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( empty ) ;
      consume the item nextc ;
    until false
  end
end
Parend
```

5. 允许生产者写时，消费者可读

关键：将生产者和消费者的互斥信号改成内部互斥信号mutexP和mutexC，其他不变

(2) 问题思考

■ 允许生产者写时，消费者可读



```
VAR mutexC, mutexP, empty, full: semaphore := 1, 1, n, 0 ;
in, out: integer := 0, 0 ;
Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;
      wait( mutexP ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( mutexP ) ;
      signal( full ) ;
    until false
  end

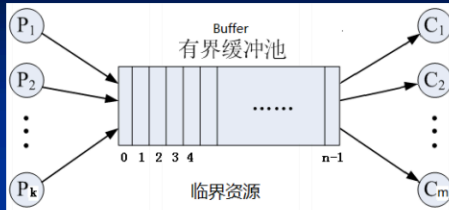
  Consumer:
  begin
    repeat
      wait( full ) ;
      wait( mutexC ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( mutexC ) ;
      signal( empty ) ;
      consume the item nextc ;
    until false
  end
end
Parend
```

6. 缓冲池无限大(?)

暂时不理解，不应该是判断full了吗？

(2) 问题思考

■ 缓冲池无限大



合肥工业大学

```
VAR mutex, full: semaphore := 1, 0 ;
    in,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( mutex ) ;
      signal( full ) ;
    until false
  end
  Consumer:
  begin
    repeat
      wait( full ) ;
      wait( mutex ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( mutex ) ;

      consume the item nextc ;
    until false
  end
Parend
```

7. 调整生产者wait顺序(?)

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
    in,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;
      wait( mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( mutex ) ;
      signal( full ) ;
    until false
  end
  Consumer:
  begin
    repeat
      wait( mutex ) ;
      wait( full ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( mutex ) ;
      signal( empty ) ;
      consume the item nextc ;
    until false
  end
Parend
```

调整了wait(mutex)和wait(full)的位置

可能导致无法判断缓冲池是否已满造成错误?

8. 调整signal顺序(?)

```
VAR mutex, empty, full: semaphore := 1, n, 0 ;
    in,out: integer := 0, 0 ;
    Buffer: array [0..n-1] of item ;
Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      wait( empty ) ;
      wait( mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      signal( mutex ) ;
      signal( full ) ;
    until false
  end
  Consumer:
  begin
    repeat
      wait( full ) ;
      wait( mutex ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      signal( empty ) ;
      signal( mutex ) ;
      consume the item nextc ;
    until false
  end
Parend
```

调整了signal(mutex)和signal(empty)的先后顺序。
未及时释放mutex，可能导致效率下降

9. 每个消费者消费每个消息1次？ 如何实现？

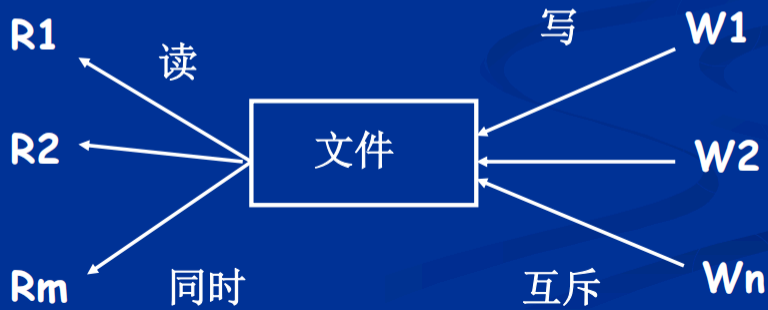
2.4.2 读者-写者问题

(1) 问题描述

多个读者、多个写者共享资源
允许多个读者同时访问
写者写时不允许其他进程读或写

(1) 读者写者问题

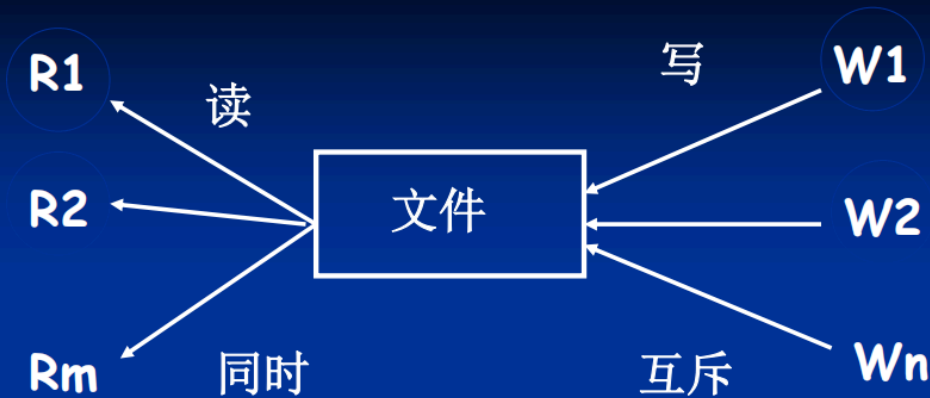
多个读者、多个写者共享资源（例如文件、数据等），允许多个读者同时访问资源，写者写时不允许其他进程读或写（互斥访问资源）。请用记录型信号量进行同步。



合肥工业大学操作系统课程组 2014

21

■ 进程资源共享关系和同步关系分析



- 读者进程：不控制
- 写者进程：互斥
- 读者写者顺序：无

(2) 解答

■ 读者-写者标准程序

```
VAR rmutex, wmutex: semaphore := 1, 1;  
    readcount: integer := 0;  
Parbegin  
  Reader:  
  begin  
    repeat  
      wait( rmutex );  
      if ( readcount = 0 ) wait( wmutex );  
      readcount := readcount + 1;  
      signal( rmutex );  
      perform read operation ;  
      wait( rmutex );  
      readcount := readcount - 1;  
      if ( readcount = 0 ) signal( wmutex );  
      signal( rmutex );  
    until false  
  end  
  Writer:  
  begin  
    repeat  
      wait( wmutex );  
      perform write operation;  
      signal( wmutex );  
    until false  
  end  
Parend
```

(2) 标准程序的2个问题

- 执行wait和signal的进程
- 临界区设置

思路

1. 设置rmutex用于读取过程中互斥访问readCount的值，readCount用于存放当前正在读取的进程个数
2. readCount=0时，Reader中可以将wait(wmutex)，从而避免此时writer写入，后正常进行读取
3. Writer在wmutex正常时可以进行写入操作

2.4.3 哲学家进餐问题

(1) 问题描述

每次每个哲学家需要拿起左右的两根筷子才能进餐

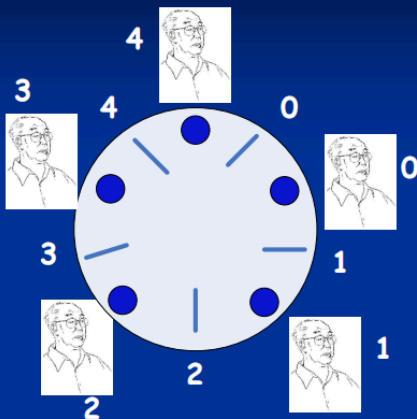
(1) 哲学家进餐问题

5位哲学家围绕圆桌而坐，反复思考和进餐。但是只有5只碗和筷子，放置如图所示，只有当哲学家同时拿起碗边的2只筷子时，才能进餐。请用记录型信号量进行同步。



(2) 解答

哲学家进餐标准程序



```
VAR chopsticks: array [0..4] of semaphore := 1,1,1,1,1;  
Parbegin  
  philosopher0:3:  
    begin  
      repeat  
        wait( chopsticks[ i ] );  
        wait( chopsticks[ (i+1) mod 5 ] );  
        eat ;  
        signal( chopsticks[ i ] );  
        signal( chopsticks[ (i+1) mod 5 ] );  
        think  
      until false  
    end  
  philosopher4:  
    begin  
      repeat  
        wait( chopsticks[ (i+1) mod 5 ] );  
        wait( chopsticks[ i ] );  
        eat ;  
        signal( chopsticks[ i ] );  
        signal( chopsticks[ (i+1) mod 5 ] );  
        think  
      until false  
    end  
Parend
```

合肥工业大学 原则：按序申请资源

31

思路 and 注意点:

- 每次需要拿两根筷子
- 对于0-3的4位哲学家，需要先拿起i的筷子，再拿i+1的筷子，后先放下i，再放下i+1
- 对于4号哲学家，按照顺序，应该先拿起i+1号，再拿i号的筷子，先放下i，再放下i+1

2.4.5 信号量集

(1) AND型信号量集机制

思想：一次性将进程需要的所有资源一起分配，使用完成后一起释放

实现：wait和signal中一次性控制所有信号量

```

type semaphore = record
  value: integer ;           // 资源数量
  L: list of process ;      // 阻塞进程队列
end

function Swait ( var S1, S2, ..., Sn : Semaphore )
Begin
  if ( S1 >= 1 and ... and Sn >= 1 )
    for i := 1 to n do
      Si := Si - 1 ;
    end
  else
    Place the process in the queue associated with the first Si found with Si < 1, and
    goto the beginning of Swait.
  end
end

function Ssignal(var S1, S2, ..., Sn : Semaphore )
Begin
  for i:=1 to n do
    Si := Si + 1
    Remove all process waiting in the queue associated with Si into the ready queue.
  end
end

```

应用：其实也就是改了一下函数和参数

(1) AND型信号量集机制

■ 应用示例1：生产者消费者问题

```

VAR mutex, empty, full: semaphore := 1, n, 0 ;
in, out: integer := 0, 0 ;
Buffer: array [0..n-1] of item ;

Parbegin
  Producer:
  begin
    repeat
      produce an item in nextp ;
      Swait( empty, mutex ) ;
      Buffer(in) := nextp ;
      in := (in + 1) mod n ;
      Ssignal( mutex, full ) ;
    until false
  end
  Consumer:
  begin
    repeat
      Swait( full, mutex ) ;
      nextc = Buffer(out);
      out := (out + 1) mod n ;
      Ssignal( mutex, empty ) ;
      consume the item nextc ;
    until false
  end
Parend

```

2.4.5 信号量集机制

(1) AND型信号量集机制

■ 应用示例2：哲学家进餐问题

```

VAR chopsticks: array [0..4] of semaphore := 1,1,1,1,1 ;
Parbegin
  philosopher:
  begin
    repeat
      Swait( chopsticks[i], chopsticks[(i+1) mod 5] ) ;
      eat ;
      Ssignal( chopsticks[i], chopsticks[(i+1) mod 5] ) ;
      think
    until false
  end
Parend

```

(2) 一般信号量集机制

思想：AND型信号量基础上，一次可申请多个单位资源（比如一次申请2个、3个单位）

实现：wait和signal中一次可以减去或加上多个单位的资源

■ 实现：

```
type semaphore = record
  value: integer ;           // 资源数量
  L: list of process ;      // 阻塞进程队列
end

function Swait ( var S1, t1, d1, S2, t2, d2, ..., Sn, tn, dn : Semaphore )
Begin
  if ( S1 >= t1 and ... and Sn >= tn )
    for i := 1 to n do
      Si := Si - di ;
    else
      Place the process in the queue associated with the first Si found with Si < ti, and
      goto the beginning of Swait.
  end
function Ssignal(var S1, d1, S2, d2, ..., Sn, dn : Semaphore )
Begin
  for i:=1 to n do
    Si := Si + di
    Remove all process waiting in the queue associated with Si into the ready queue.
  end
```

应用：一次分配多个读者进程

(2) 一般信号量集机制

■ 应用实例：读者写者问题

RN: 允许读者进程上限

```
VAR RN: integer ;
    L,mutex: semaphore := RN, 1 ;
Parbegin
  Reader:
  begin
    repeat
      Swait(L,1,L, mutex,1,0) ;
      perform read operation ;
      Ssignal(L, 1) ;
    until false
  end
  Writer:
  begin
    repeat
      Swait( mutex,1,1, L,RN,0) ;
      perform write operation;
      Ssignal( mutex,1) ;
    until false
  end
Parend
```

2.5 进程通信

2.5.0 定义

进程通信的定义：进程间的信息交换

- 交换的信息少者是一个状态或数值
- 多者是成千上万字节

2.5.1 共享存储器系统

1. 基本思想

- 相互通信的进程共享数据结构或共享存储区
- 进程间通过这些空间进行通信

2. 实现过程

1. 进程申请一片内存，拿到内存描述符；如果已分配过，直接拿到内存描述符
2. 内存描述符连接内存到本进程
3. 读写

3. 共享存储区的访问同步

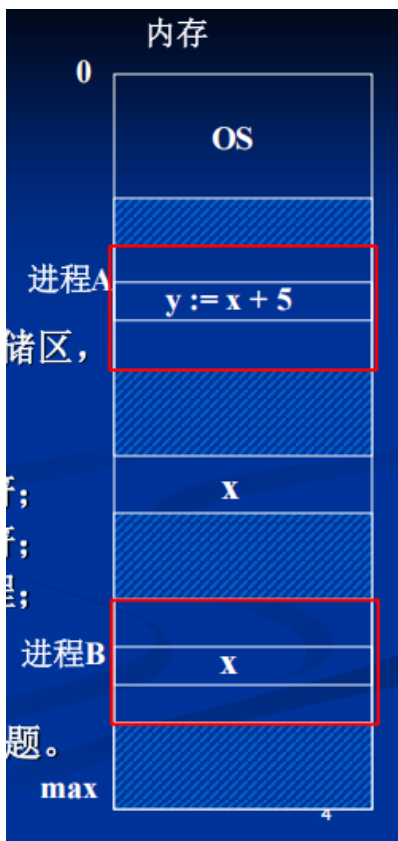
- 共享存储区作为资源，需要考虑共享问题（多个进程同时访问如何处理）

4. 分类

- 基于共享数据结构的通信方式
- 基于共享存储区的通信方式

5. 特点

- 适合大量的数据快速交换（空间距离近）
- 程序员负担重（设计实现复杂）



2.5.2 消息通信系统

(0) 基本定义

1. 基本思想

进程间的数据交换，是以格式化的消息为单位进行通信

2. 消息通信方式分类

- **直接通信**：信息直接传递给接收方
 - 发送方 → 接收方
 - 发送时，指定接收方（可指定多个或广播式地址）的地址或标识
 - 接收时，允许接收任意发送方消息，读取消息同时获取发送方地址
- **间接通信**：借助收发双方进程之外的共享数据结构作为通信中转，如消息队列
 - 发送方 → 共享数据结构（通信中转） → 接收方
 - 发送方和接收方的数量可以是任意的

(1) 直接通信方式

OS中存在两个通信原语：

1. **发送进程**：Send(receiver, msg)
2. **接收进程**：Receive(sender, msg)

图中，进程X将消息a发送给进程Y

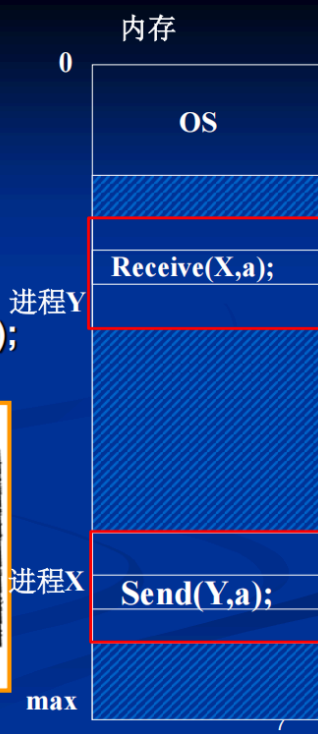
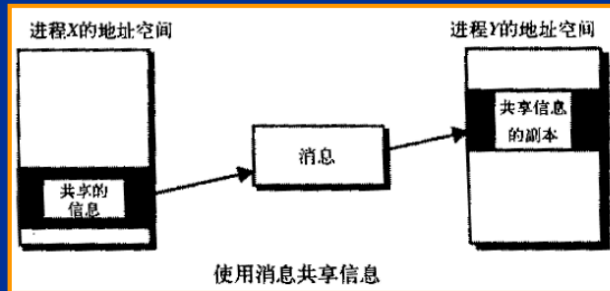
2.5 进程通信

2.5.1 消息通信系统

(1) 直接通信方式

OS提供两个通信原语:

- ① 发送进程: **Send(receiver,msg);** 进程Y
- ② 接收进程: **Receive(sender,msg);**



合肥工业大学操作系统课程组 2014

(2) 间接通信方式 (使用信箱通信)

1. 使用信箱进行通信的原理

使用**信箱**通信的通信原语:

1. **发送进程**: **Send(mailbox, msg)**
2. **接收进程**: **Receive(mailbox, msg)**

信箱作为一种**共享数据结构**, 作为**通信的中介**

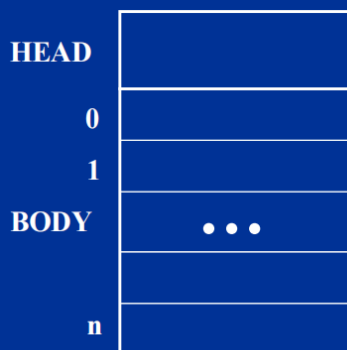
2.5 进程通信

2.5.2 消息通信系统

(2) 间接通信方式:信箱通信

- ① 发送进程: **Send(mailbox,msg);** 进程A
- ② 接收进程: **Receive(mailbox,msg);**

■ 信箱 (mailbox)



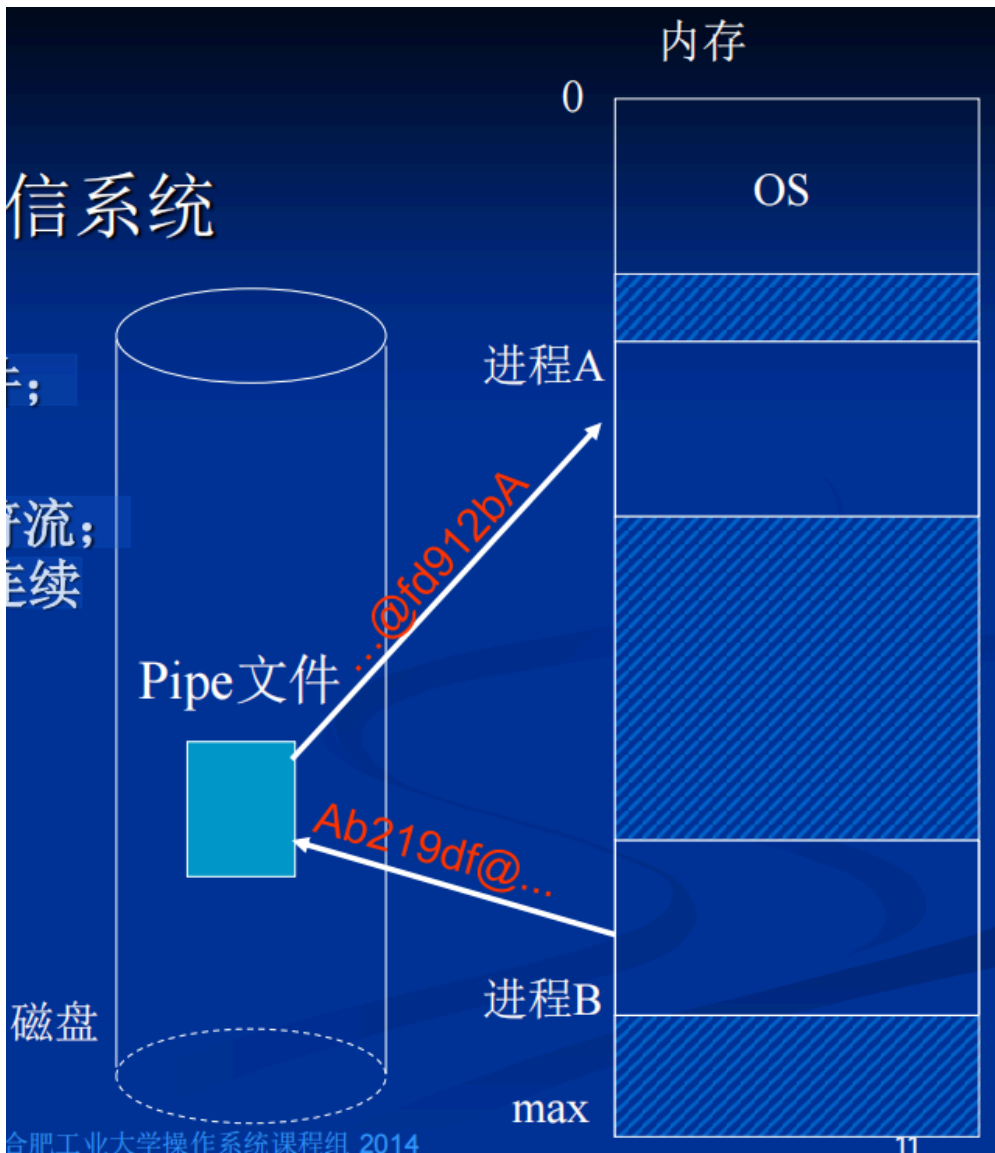
合肥工业大学操作系统课程组 2014

2. 信箱

- **信箱的分类**
 - 公有信箱
 - 私有信箱
 - 共享信箱
- **信箱的特点**
 - 操作系统隐藏通信细节；
 - 通信程序简单
 - 可以跨网络传输；
 - 灵活
- **进程同步**
 - 发送阻塞、接收阻塞
 - 发送不阻塞、接收阻塞
 - 发送不阻塞、接收不阻塞

2.5.3 管道 (Pipe) 通信系统

1. **基本思想**
 - 在磁盘创建一个Pipe文件，大小固定，如4kB；
 - 发送进程写连续字符流；
 - 接收进程按顺序读连续字符流；
 - 本质就是创建了一个通信的中介介质文件，用于进程间的字符传输



2. 实现技术

- 对管道文件的共享：同步、互斥
- UNIX首创

3. Windows操作系统的管道通信

- Windows提供**无名管道**和**命名管道**两种管道机制。
- 利用CreatePipe可创建**无名管道**并得到两个读写句柄；
- 利用ReadFile和WriteFile可**并行无名管道的读写**。

2.6 线程-函数级并发执行

2.6.1 线程（Thread）的基本概念

(1) 线程的引入

1. 传统线程的两个基本属性

- 拥有**资源**的独立单位
- **调度和分派**的基本单位

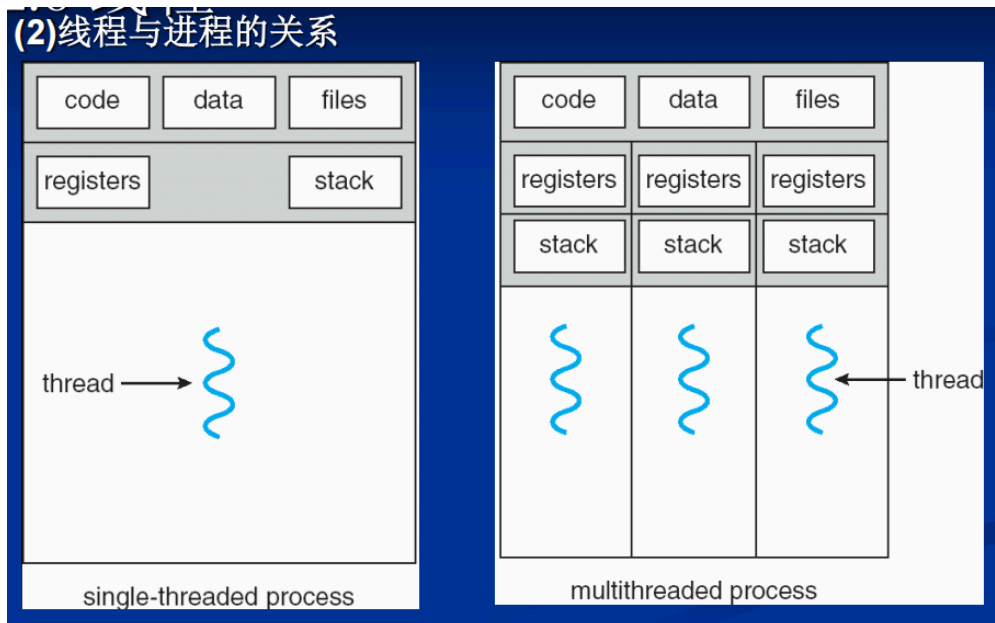
2. 传统线程的缺点

- 管理和运行资源开销大
- 限制并发度，多道程序设计道数不能太高

3. 线程的概念（特点）

- 轻量级进程，进程中的一个轻型运行实体
- 一个CPU调度和分派的基本单位
- 可并发执行
- 共享进程资源

(2) 线程与进程的关系



从图中可以看出

- 单线程进程享受所有资源；
- 多线程进程各线程有独立的寄存器和栈，而指令数据文件都是相同的

(3) 线程的开销

- 线程控制块（TCB）
- 寄存器状态，堆栈，局部变量拷贝
- 线程运行状态（就绪、阻塞、执行）
- 可以创建、撤消另一个线程
- 优先级与信号屏蔽

(4) 引入进程的好处

1. 创建一个新线程花费时间少（结束亦如此）
2. 两个线程的切换花费时间少

3. 同一进程内的线程共享内存和文件，因此它们之间相互通信无须调用内核
4. 适合多处理机系统

2.6.2 线程的实现方法

(1) 内核支持线程

1. 实现方法：
 - 使用内核支持线程运行
 - 内核中各线程均有线程控制块(TCB)，可用于控制
 - 设置内核支持线程，则以线程为单位进行
 - 轮转法调度时，各线程轮流执行一个时间片
2. 优点
 - 对多处理器，核心可以同时调度同一进程的多个线程
 - 阻塞是在线程一级完成
 - 核心例程是多线程的
3. 缺点

在同一进程内的线程切换调用内核，导致速度下降
4. 举例

Windows 2000/XP、Macintosh和OS/2操作系统

(2) 用户级线程

1. 实现方法
 - 仅存在于用户空间中
 - 可为一个应用程序建立多个用户级线程
 - 线程控制块(TCB)在用户空间，用户来维护
 - 线程的创建、撤销、状态转换等，无法使用系统调用实现
 - 用户级线程的切换（包括同步与通信），如果在一个应用进程的许多进程中，无需内核支持
 - 使用专门的线程库管理线程，无需内核帮助，内核也不知道用户级线程
 - 设置用户级线程系统，调度仍然以进程为单位（注意别看错）
 - 轮转法调度时，各进程轮流执行一个时间片
2. 优点
 - 线程切换不调用核心
 - 调度是应用程序特定的：可以选择最合适(好)的算法
 - ULT可运行在任何操作系统上（只需要线程库）
3. 缺点

- 大多数系统调用是阻塞的，因此核心阻塞进程，故进程中所有线程将被阻塞
- 核心只将处理器分配给进程，同一进程中的两个线程不能同时运行于两个处理器上